📡 Events Service

> 🚀 **Built for:** Real-time event monitoring, user notifications, and alert workflows
> 🔧 **Tech Stack:** GraphQL, AWS Lambda, AWS IAM, Amazon Cognito
> 📊 **Data Sources:** Amazon Pinpoint, Amazon Kinesis Firehose, Amazon S3

🏷️ Enums

📊 EventType

Event classification types:

Value	Description	Category
`SENSOR`	📡 Sensor-based events	Hardware
`GENERIC`	🔧 Generic system events	System

🔄 EventState

Event state management:

Value	Description	Status
`ACTIVE`	✅ Event is active and requires attention	Active
`INACTIVE`	❌ Event has been	Resolved

On this page

- 🏷️ Enums
  - 📊 EventType
  - 🔄 EventState
  - ⚠️ EventSeverity
  - 📱 NotificationType
  - 📊 NotificationState
- 🔵 GraphQL
  - 🔍 Queries
    - 📡 `devicesEventsList`
    - 🏢 `organizationsEventsList`
    - 📋 `eventsMetadataList`
    - 🔔 `notificationsSubscriptionsList`
  - 🖍️ Mutations
    - ❌ `eventsDeactivate`
    - ➕ `notificationsSubscriptionsCreate`
    - 🗑️ `notificationsSubscriptionsRemove`
  - 💻 HTTP Request Examples
    - 🟧 Using JavaScript/fetch
    - 🐍 Using Python/requests
    - 🔧 Using cURL
  - 📡 Subscriptions
    - 🔧 **Setup Requirements**
    - 🔔 `eventsFeed`
    - 📊 Subscription Examples
    - 🟧 Using JavaScript/Node.js
  - 📋 Types
    - 📊 Event
    - 📋 EventMetadata
    - 🔔 NotificationSubscription
    - 📊 TimePeriod
    - 📋 SubscriptionDetails

	resolved or deactivated	

⚠️ EventSeverity

Event severity levels:

Value	Description	Priority
`LOW`	🟢 Low priority event	Low
`MEDIUM`	🟡 Medium priority event	Medium
`HIGH`	🔴 High priority event	High

📱 NotificationType

Notification delivery methods:

Value	Description	Channel
`SMS`	📱 SMS text message	Mobile
`EMAIL`	📧 Email notification	Email
`PUSH`	🔔 Push notification	Mobile App

📊 NotificationState

Notification delivery status:

Value	Description	St
`OK`	✅ Notification delivered successfully	Su
	❌ Deli	

`MISSING_ENDPOINT`	❌ Delivery endpoint not configured	Fa

🔵 GraphQL

The Events Service provides GraphQL queries, mutations, and subscriptions for event monitoring and notifications.

Note: All GraphQL requests require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API Overview section for authentication setup.

Base URL: Get the GraphQL endpoint from the endpoints configuration: `endpoints.GraphQL.events.https`

🔍 Queries

📡 `devicesEventsList`

> Lists events for a specific device.

Arguments:

Parameter	Type	Required
`deviceId`	`ID!`	✅
`period`	`TimePeriod`	🟣
`nextToken`	`String`	🟣

Returns: `Events` - List of device events with pagination

Example:

```
query GetDeviceEvents {
  devicesEventsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    period: {
      startTimestamp: "1735689600000"
      endTimestamp: "1735776000000"
    }
  ) {
    events {
      deviceId
      timestamp
      eventId
      type
      eventState
      severity
      sensorName
      sensorValue
      displayName
    }
    nextToken
  }
}
```

🏢 organizationsEventsList

> Lists events for all devices in an organization.

Arguments:

Parameter	Type	Requ
organizationId	ID!	✅
period	TimePeriod	🟣
nextToken	String	🟣

Returns: `Events` - List of organization events with pagination

Example:

```
query GetOrganizationEvents {
```

```
  organizationsEventsList(
    organizationId: "ORG-PROD-001"
    period: {
      startTimestamp: "1735689600000"
      endTimestamp: "1735776000000"
    }
  ) {
    events {
      deviceId
      timestamp
      eventId
      type
      severity
      displayName
    }
    nextToken
  }
}
```

📋 `eventsMetadataList`

> Lists available event metadata definitions.

Arguments:

Parameter	Type	Required	D
`nextToken`	`String`		P t

Returns: `EventMetadataList` - List of event metadata with pagination

Example:

```
query GetEventMetadata {
  eventsMetadataList {
    eventMetadataItems {
      eventId
      name
      description
    }
    nextToken
  }
}
```

🔔 `notificationsSubscriptionsList`

> Lists notification subscriptions for a

> Lists notification subscriptions for a user.

Arguments:

Parameter	Type	Required
`userId`	`ID!`	✅
`organizationId`	`ID!`	✅

Returns: `NotificationSubscriptions` - List of notification subscriptions

Example:

```
query GetNotificationSubscriptions {
  notificationsSubscriptionsList(
    userId: "user-abc-123-def-456"
    organizationId: "ORG-PROD-001"
  ) {
    subscriptions {
      id
      userId
      organizationId
      eventIds
      type
      state
    }
  }
}
```

✏️ Mutations

❌ `eventsDeactivate`

> Deactivates an event (marks as resolved).

Arguments:

Parameter	Type	Requi
`payload`	`EventPayload!`	✅

Returns: `Event` - Deactivated event

Example:

```
mutation DeactivateEvent {
  eventsDeactivate(payload: {
    deviceId: "DEVICE-ABC123-XYZ789"
    timestamp: "1735689600000"
    eventId: "event-cloud-connected"
    eventState: INACTIVE
    organizationId: "ORG-PROD-001"
    type: GENERIC
    severity: MEDIUM
    displayName: "Device Connected"
  }) {
    deviceId
    eventId
    eventState
    updatedTimestamp
    organizationId
  }
}
```

`notificationsSubscriptionsCreate`

> Creates a new notification subscription.

Arguments:

Parameter	Type
`subscriptionDetails`	`SubscriptionDe`

Returns: `NotificationSubscription` - Created subscription

Example:

```
mutation CreateNotificationSubscripti
  notificationsSubscriptionsCreate(su
    type: EMAIL
    userId: "user-abc-123-def-456"
    owningOrganization: "ORG-PROD-001
    organizationId: "ORG-PROD-001"
    eventId: "event-cloud-connected"
```

```
  }) {
    id
    userId
    organizationId
    eventIds
    type
    state
  }
}
```

> **Note:** Subscriptions are managed per user and organization. Creating duplicate subscriptions for the same event may return service errors indicating existing coverage.

🗑️ `notificationsSubscriptionsRemove`

> Removes a notification subscription.

Arguments:

Parameter	Type	Required
`userId`	`ID!`	✅
`subscriptionId`	`ID!`	✅

Returns: `UnsubscribeInfo` - Unsubscription details

Example:

```
mutation RemoveNotificationSubscripti
  notificationsSubscriptionsRemove(
    userId: "user-abc-123-def-456"
    subscriptionId: "sub-abc-123-def-
  ) {
    subscription {
      id
      userId
      organizationId
      eventIds
      type
      state
```

```
    }
  }
}
```

💻 HTTP Request Examples

The following examples show how to make GraphQL queries and mutations using HTTP. Each example includes the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section.

🟧 Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

// Example query
const query = `
  query GetEvents {
    devicesEventsList(
      deviceId: "DEVICE-ABC123-XYZ789
      period: {
        startTimestamp: "173568960000
        endTimestamp: "1735776000000"
      }
    ) {
      events {
        deviceId
        timestamp
        eventId
        type
        eventState
        severity
      }
      nextToken
    }
  }
`;
```

```
const graphqlResponse = await fetch(g
  method: 'POST',
  headers: {
    'Content-Type': 'application/jso
    'Authorization': `Bearer ${idToke
  },
  body: JSON.stringify({ query })
});

const data = await graphqlResponse.js
console.log(data);
```

Using Python/requests

```
import requests

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

# Example query
query = """
query GetEvents {
  devicesEventsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    period: {
      startTimestamp: "1735689600000"
      endTimestamp: "1735776000000"
    }
  ) {
    events {
      deviceId
      timestamp
      eventId
      type
      eventState
      severity
    }
    nextToken
  }
}
"""

response = requests.post(
    graphql_endpoint,
    headers={
        'Content-Type': 'application/
        'Authorization': f"Bearer {id
```

```
    },
    json={'query': query}
)

data = response.json()
print(data)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_GENERICS=$(echo "$ENDPOINTS"
GRAPHQL=$(echo "$ENDPOINTS" | jq -r '

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j

QUERY='{"query":"query GetEvents {\\n

curl -sS -H "Content-Type: applicatio
  -H "Authorization: Bearer $ID_TOKEN
  -X POST "$GRAPHQL" \
  --data "$QUERY" | jq '.'
```

📡 Subscriptions

> ⚠️ **Important**: Subscriptions are more complex than queries/mutations as they require WebSocket connections.

🔧 Setup Requirements

JavaScript/Node.js:

```
npm install aws-appsync aws-appsync-a
```

Note: `aws-appsync` requires `graphql` version 14.x or 15.0.0–15.3.0 (not 16+). If you encounter compatibility issues, install with: `npm install aws-appsync aws-appsync-auth-link graphql@14 graphql-tag`

Pvthon:

```
pip install requests websocket-client
```

Note: Use the Cognito IdToken obtained via the REST API (see API Overview). Get endpoints from the Endpoints API; the client URL comes from `endpoints.GraphQL.events.https`. The `receiver` must be the JWT claim `cognito:username` from your IdToken.

🔔 eventsFeed

> Real-time feed of event notifications.

Arguments:

Parameter	Type	Required	De
`receiver`	`ID!`	✅	The the

Returns: `EventNotice` - Real-time event notifications

Example:

```
subscription EventUpdates {
  eventsFeed(receiver: "user-abc-123-
    receiver
    item {
      deviceId
      timestamp
      eventId
      type
      eventState
      severity
      sensorName
      sensorValue
      displayName
    }
  }
}
```

📊 Subscription Examples

The following examples show complete subscription setup including authentication. For detailed authentication setup and token management, see the API Overview section.

Using JavaScript/Node.js

```
import { AWSAppSyncClient } from "aws
import { AUTH_TYPE } from "aws-appsyn
import gql from "graphql-tag";

// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

// Extract user ID from JWT token (re
const tokenPayload = JSON.parse(Buffe
const userId = tokenPayload['cognito:

// Create AppSync client
const client = new AWSAppSyncClient({
  url: graphqlEndpoint,
  region: "eu-central-1",
  auth: {
    type: AUTH_TYPE.AWS_LAMBDA,
    token: async () => `Bearer ${idTo
  },
  disableOffline: true,
});

// Subscribe to events feed
const subscription = client.subscribe
  query: gql`
    subscription EventsFeed {
      eventsFeed(receiver: "${userId}
        receiver
        item {
          deviceId
          timestamp
          eventId
          type
          eventState
          severity
          sensorName
```

```
          sensorValue
          displayName
        }
      }
    }
  `
});

subscription.subscribe({
  next: (data) => console.log("Receiv
  error: (error) => console.error("Er
});
```

Using Python/requests

```
import json
import base64
import websocket
import requests
from uuid import uuid4

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

def header_encode(header_obj):
    """Encode header using Base 64"""
    return base64.b64encode(json.dump

# Extract user ID from JWT token
token_payload = json.loads(base64.b64
user_id = token_payload['cognito:user

# Build WebSocket URL and host
wss_url = graphql_endpoint.replace('h
host = graphql_endpoint.replace('http

# Generate subscription ID
sub_id = str(uuid4())

# Create JWT authentication header
auth_header = {
    'host': host,
    'Authorization': f"Bearer {id_tok
}

# GraphQL subscription
gql_subscription = json.dumps({
```

```
    'query': f'subscription EventsFee
    'variables': {}
})

# WebSocket event callbacks
def on_message(ws, message):
    message_object = json.loads(messa
    message_type = message_object['ty

    if message_type == 'connection_ac
        # Register subscription
        register = {
            'id': sub_id,
            'payload': {
                'data': gql_subscript
                'extensions': {'autho
            },
            'type': 'start'
        }
        ws.send(json.dumps(register))

    elif message_type == 'start_ack':
        print("✅ Subscription regis

    elif message_type == 'data':
        print("✅ Received subscripti
        # Stop subscription
        ws.send(json.dumps({'type': '

def on_open(ws):
    ws.send(json.dumps({'type': 'conn

# Create WebSocket connection
connection_url = wss_url + '?header='
ws = websocket.WebSocketApp(
    connection_url,
    subprotocols=['graphql-ws'],
    on_open=on_open,
    on_message=on_message
)

# Run WebSocket (use proper threading
ws.run_forever()
```

📋 Types

📊 Event

> Event representation

Field	Type
deviceId	ID!
timestamp	String!

timestamp	String!
eventId	ID
organizationId	ID
updatedTimestamp	String
type	EventType
eventState	EventState
severity	EventSeverity
sensorName	String
sensorValue	Float
metadata	String
displayName	String

📋 EventMetadata

> Event metadata definition

Field	Type	Required
eventId	ID!	✅
name	String	🟣
description	String	🟣

🔔 NotificationSubscription

> Notification subscription

Field	Type

id	ID!
userId	ID!
organizationId	ID!
eventIds	[ID!]!
type	NotificationType!
state	[NotificationState!

📊 TimePeriod

Time range filter

Field	Type	Require
startTimestamp	String!	✅
endTimestamp	String!	✅

📋 SubscriptionDetails

Subscription creation details

Field	Type
type	NotificationTyp
userId	ID!
owningOrganization	ID
organizationId	ID
eventId	ID

deviceId	ID

📊 Events

Paginated events list

Field	Type	Required
events	[Event!]!	✅
nextToken	String	

📋 EventMetadataList

Paginated event metadata list

Field	Type
eventMetadataItems	[EventMetadata!
nextToken	String

📦 NotificationSubscriptions

Notification subscriptions list

Field	Type
subscriptions	[NotificationSubscri
nextToken	ID

📢 EventNotice

Event notification

Field	Type	Required	De
receiver	ID!	✅	Re ide
item	Event!	✅	Ev

🗑️ UnsubscribeInfo

Unsubscribe information

Field	Type
subscription	NotificationSubscript

📋 Sample Responses

✅ Successful Query Response


```
{
  "data": {
    "devicesEventsList": {
      "events": [
        {
          "deviceId": "DEVICE-ABC123-
          "timestamp": "1735689600000
          "eventId": "event-cloud-con
          "type": "GENERIC",
          "eventState": "ACTIVE",
          "severity": "MEDIUM",
          "displayName": "Device Conn
        }
      ],
      "nextToken": "eyJsYXN0RXZhbHVhd
    }
  }
}
```


❌ Error Response


```
{
  "errors": [
    {
      "message": "Event not found",
      "locations": [{"line": 3, "colu
      "path": ["devicesEventsList"],
      "extensions": {
        "code": "EVENT_NOT_FOUND",
```

```
        "exception": {
          "stacktrace": ["Error: Even
        }
      }
    }
  ],
  "data": null
}
```

📄 Pagination

For paginated results, use the `nextToken` from the response in subsequent requests:

```
query GetMoreEvents {
  devicesEventsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    period: {
      startTimestamp: "1735689600000"
      endTimestamp: "1735776000000"
    }
    nextToken: "eyJsYXN0RXZhbHVhdGVkS
  ) {
    events {
      deviceId
      timestamp
      eventId
      type
      severity
    }
    nextToken
  }
}
```

🌎 Configuration Reference: Harvia Endpoints API

📝 Note: Always fetch the latest configuration to ensure you're using the current endpoints, regions, and client IDs. The configuration may change over time.

🌐 REST API

The REST API provides a subset of the

full Events Service GraphQL API for common integration flows.

Note: All REST API endpoints require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API Overview section for authentication setup.

Base URL: Get the REST API base URL from the endpoints configuration: `endpoints.RestApi.events.https`

`GET /events/events`

> Retrieves a paginated list of events for a specific device and cabin within a time range.

Arguments:

Parameter	Type	Require
`deviceId`	`string`	✅
`cabinId`	`string`	
`cabinName`	`string`	
`startTimestamp`	`string`	✅

endTimestamp	string	✅
limit	integer	
order	string	
nextToken	string	

Notes:

- If neither `cabinId` nor `cabinName` is provided, the Events service defaults to cabin `C1`.
- If `shadowName` is returned, the response is already filtered to that cabin/sub-shadow.

Success Response (200):

```
{
  "deviceId": "ABC123",
  "shadowName": "C1",
  "events": [
    {
      "eventId": "event-overheat-warn
      "deviceId": "ABC123",
      "timestamp": "1735689600000",
      "updatedTimestamp": "1735689660
      "type": "SENSOR",
      "eventState": "ACTIVE",
      "severity": "HIGH",
      "sensorName": "temperature",
      "sensorValue": 92.4,
```

```
      "displayName": "Overheat warnir
      "subId": "C1"
    }
  ],
  "nextToken": "eyJwYWdlIjoiMiJ9"
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

💡 REST API Examples

Each example below shows the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section.

🟧 Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiBase = endpoints.RestApi
const restApiGenerics = endpoints.Res

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;
const start = `${Date.now() - 60 * 60
const end = `${Date.now()}`;

const events = await fetch(
  `${restApiBase}/events/events?devic
  {
    headers: { Authorization: `Bearer
  }
).then(r => r.json());

console.log(events);
```

Using Python/requests

```
import requests
import time

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_base = endpoints["RestApi"][
rest_api_generics = endpoints["RestAp

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]
end = int(time.time() * 1000)
start = end - 60 * 60 * 1000

events = requests.get(
    f"{rest_api_base}/events/events",
    headers={"Authorization": f"Beare
    params={
        "deviceId": "ABC123",
        "cabinId": "C1",
        "startTimestamp": start,
        "endTimestamp": end,
        "limit": 50,
        "order": "DESC",
    },
).json()

print(events)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_BASE=$(echo "$ENDPOINTS" | j
REST_API_GENERICS=$(echo "$ENDPOINTS"

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j
END_MS=$(date +%s000)
START_MS=$((END_MS - 3600000))

curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/events/events?de
```